FITTRACK: A COLLABORATIVE MULTI-TENANT FITNESS & NUTRITION PLATFORM WITH PERSISTENT AI HEALTH COACHING MEMORY

Submitted by

VINOD BALASUBRAMANIAM – 241701065
PREM SRINIVAS – 241701044

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND DESIGN





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE
ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this report titled "FITTRACK: A COLLABORATIVE MULTI-TENANT FITNESS & NUTRITION PLATFORM WITH PERSISTENT AI HEALTH COACHING MEMORY" is the bonafide work of **VINOD BALASUBRAMANIAM** and **PREM SRINIVAS**, who carried out the work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Certified further that this project contributes to SDG 3 – Good Health and Well-Being, SDG 4 – Quality Education, SDG 9 – Industry, Innovation and Infrastructure, and SDG 10 – Reduced Inequalities, with a primary focus on SDG 3 – Good Health and Well-Being, through the development of an accessible, data-driven, and AI-assisted health and fitness management platform.

Signature of the HOD
Dr. S. Vinodh Kumar
Professor & Head of the Department

Signature of the Supervisor
Dr. P. Revathy
Professor

ABSTRACT

Contemporary health and fitness tracking applications operate within fragmented, single-user, and session-bound paradigms. Standard fitness management software executes without long-term retention of user dietary restrictions, past injury records, micro-nutrient tendencies, or personal biomechanical constraints across multi-turn interactions. Furthermore, modern wellness platforms lack real-time synchronization between personal trainers, nutritionists, and an autonomous AI health coach. Mobile extensions isolate tracking to local offline client storage, fitness cloud tools relegate AI to isolated sidebars, and autonomous coaching agents execute without real-time collaborative oversight from human health professionals.

This project presents **FitTrack**, a browser-native collaborative multi-tenant fitness and nutrition web platform engineered with persistent health memory. FitTrack addresses cross-session context loss and client-trainer isolation through two core architectural mechanisms: a Dual-Tier Persistent Memory System utilizing an ACID-compliant relational SQLite store (user_memories and workout_logs) to retain metabolic profiles, injury history, workout preferences, and historical macro logs, alongside Multiplayer WebSocket Room Synchronization using a FastAPI real-time orchestration hub connecting clients and personal trainers to a unified AI coaching session.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. S. VINODHKUMAR, Ph.D., Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Dr. P. REVATHY, Department of Computer Science and Design, for her valuable guidance throughout the course of the project. We are very glad to thank our Project Coordinator, Dr. S.P. KARUPPIAH, Ph.D., Department of Computer Science and Design, for his valuable guidance and useful tips during our reviews to build our project.

VINOD BALASUBRAMANIAM (241701065)
PREM SRINIVAS (241701044)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	BONAFIDE CERTIFICATE	**2**
	ABSTRACT	**3**
	ACKNOWLEDGEMENT	**4**
	LIST OF TABLES	**7**
	LIST OF FIGURES	**8**
	LIST OF ABBREVIATIONS	**9**
1.	**INTRODUCTION**	**10**
	1.1 GENERAL	**10**
	1.2 BACKGROUND	**11**
	1.3 PROBLEM STATEMENT	**12**
	1.4 MOTIVATION	**12**
	1.4.1 THE IMPERATIVE FOR SHARED CONTEXT IN HEALTH AND WELLNESS	**12**
	1.4.2 INSTITUTIONAL HEALTH CAPITAL & CROSS-SESSION MEMORY PRESERVATION	**13**
	1.4.3 BROWSER-NATIVE ACCESSIBILITY & DEMOCRATIZATION OF HIGH-QUALITY COACHING	**14**
	1.4.4 CONFLICT-RESILIENT COLLABORATION VIA OPTIMISTIC CONCURRENCY CONTROL	**15**
	1.5 OBJECTIVES	**15**
2.	**LITERATURE REVIEW**	**17**
	2.1 INTRODUCTION	**17**
	2.2 REVIEW OF RELATED WORK	**17**
	2.3 COMPARATIVE ANALYSIS	**20**
	2.4 COMPARATIVE ANALYSIS OF THE PROPOSED SYSTEM	**21**
	2.5 RESEARCH GAP	**21**
3.	**THEORETICAL BACKGROUND AND SYSTEM DESIGN**	**22**
	3.1 ARCHITECTURAL OVERVIEW	**22**
	3.2 DUAL-TIER RELATIONAL HEALTH MEMORY	**23**
	3.3 RELATIONAL DATABASE SCHEMA	**23**
4.	**SYSTEM IMPLEMENTATION AND PROJECT DESCRIPTION**	**24**
	4.1 COMPREHENSIVE TECHNOLOGY STACK & ARCHITECTURAL RATIONALE	**24**
	4.1.1 OVERVIEW OF THE SELECTED TOOLCHAIN	**24**
	4.1.2 DEEP-DIVE LAYER BREAKDOWN & ENGINEERING JUSTIFICATION	**24**

CHAPTER NO.	TITLE	PAGE NO.
	4.2 FUNCTIONAL VERIFICATION SUITE & SYSTEM TESTING STRATEGY	28
	4.2.1 VERIFICATION FRAMEWORK & TESTING METHODOLOGY	28
	4.2.2 DETAILED 17-TEST AUTOMATED VERIFICATION MATRIX	29
	4.2.3 DEEP-DIVE TEST ANALYSIS & VERIFICATION MECHANISMS	30
5.	**CONCLUSIONS AND FUTURE ENHANCEMENTS FOR PHASE II**	**33**
	5.1 EXECUTIVE SUMMARY & PHASE-I ACCOMPLISHMENTS	33
	5.2 DETAILED ARCHITECTURAL LIMITATIONS OF PHASE I	34
	5.3 COMPREHENSIVE PHASE-II OBJECTIVES & STRATEGIC ROADMAP	35
	5.4 PHASE-II ENGINEERING WORK SCHEDULE	37
	REFERENCES	**38**
	PROGRAM OUTCOMES	**39**
	PROGRAM SPECIFIC OUTCOMES	**40**

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Literature survey comparison	20
Table 2.2	Comparative analysis of FitTrack with related work	21
Table 3.1	Relational database schema	23
Table 4.1	Technology stack and architectural rationale	24
Table 4.2	Automated verification test matrix	29
Table 5.1	Phase II engineering work schedule	37

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	FitTrack persistent memory pipeline	14
Figure 5.1	FitTrack Phase II architecture	35

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BMR	Basal Metabolic Rate
CRDT	Conflict-Free Replicated Data Type
CSS	Cascading Style Sheets
E2E	End-to-End
HMR	Hot Module Replacement
HRV	Heart Rate Variability
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
OCC	Optimistic Concurrency Control
OT	Operational Transformation
RAG	Retrieval-Augmented Generation
ReAct	Reason and Act
REST	Representational State Transfer
SDG	Sustainable Development Goal
SQL	Structured Query Language
SVG	Scalable Vector Graphics
TDEE	Total Daily Energy Expenditure
UI	User Interface
WAL	Write-Ahead Logging
Wasm	WebAssembly

CHAPTER 1

INTRODUCTION

1.1 General

Modern personal health management has evolved into a data-intensive discipline where fitness progress depends on consistent tracking, precise biological metrics monitoring, and expert human guidance. Concurrently, recent breakthroughs in generative artificial intelligence and Large Language Models (LLMs) have transformed personalized meal plan generation, automated macro calculation, and dynamic exercise routine synthesis. Despite these parallel technological advancements, a critical structural limitation persists across contemporary digital health tools: modern fitness applications treat wellness as an isolated, single-user task. When an individual interacts with a mobile calorie tracker or an AI workout generator, the resulting dietary adjustments, historical injury logs, and health preferences remain siloed within local device caches or session-bound chat interfaces. Consequently, human personal trainers, nutritionists, and health coaches are deprived of real-time visibility into their clients' daily adherence and AI-driven recommendations, leading to fragmented coaching strategies, lost historical context, and potential safety risks. FitTrack directly resolves this systemic disconnect by embedding an autonomous, multi-turn AI health agent into a collaborative, browser-native web dashboard where clients, personal trainers, and the AI agent share a unified, persistent health context. Rather than functioning as an ephemeral chat sidebar or an offline tracking buffer, the FitTrack AI agent operates as an active, shared teammate within a synchronized client project room. Through full-duplex WebSocket channels and a dual-tier relational health memory architecture, FitTrack permanently retains metabolic targets, injury histories, dietary restrictions, and historical workout logs across sessions. This allows both human trainers and the AI agent to co-steer personalized workout regimens, prevent write conflicts through Optimistic Concurrency Control (OCC), and deliver

data-driven, continuous wellness management without requiring repetitive manual context re-entry.

1.2 Background

The evolution of modern digital health tools is driven by two intersecting technological transitions:

1. **Shift to Cloud-Native Real-Time Synchronization:** Personal health management has rapidly evolved from manual, paper-based tracking and local smartphone databases to cloud-hosted, real-time web architecture. Modern cloud platforms enable clients, personal trainers, and nutritionists to access synchronized health telemetry—including daily calorie targets, macronutrient splits, dynamic exercise logs, and biometric metrics—from any location without requiring complex client-side application installations.
2. **Evolution of Generative AI in Health Coaching:** Simultaneously, artificial intelligence in the wellness sector has transitioned from static, heuristic-based macro calculators to autonomous, multi-turn AI health coaching agents. Operating under agentic paradigms such as ReAct (Reason and Act), modern language models do not merely generate static meal recipes or workout templates; they actively evaluate user biometric history, analyze daily adherence trends, update health memory records, and iterate on structured fitness recommendations.

Despite these advancements, existing cloud fitness platforms and autonomous AI coaching agents have developed along separate trajectories. Cloud fitness platforms provide multi-user access for human trainers and clients but relegate AI functionality to stateless, single-session chat sidebars. Conversely, autonomous health agents run as isolated background processes or single-tenant mobile utilities, depriving personal trainers of real-time visibility, collaborative intervention capabilities, and shared institutional memory. FitTrack synthesizes these two trajectories into a single, cohesive full-stack platform. By pairing a browser-native React client interface with

an asynchronous FastAPI WebSocket orchestration gateway, an LiteLLM-routed ReAct AI coaching engine, an ACID-compliant dual-tier relational database, and file/record-level Optimistic Concurrency Control (OCC), FitTrack establishes a real-time collaborative workspace where human health professionals and autonomous AI agents co-steer client wellness safely and transparently.

1.3 Problem Statement

Existing fitness applications suffer from two major deficits: the Episodic-Temporal Deficit (where AI coaches forget injury history, dietary allergies, and past milestones upon starting a new session) and the Single-User Isolation Paradigm (where personal trainers cannot observe AI-generated plan alterations or real-time user workout logs simultaneously).

1.4 Motivation

1.4.1 The Imperative for Shared Context in Health and Wellness

The architectural design and full-stack engineering of the FitTrack platform are driven by the imperative operational requirement for a unified health workspace where clients, human personal trainers, and autonomous AI health agents share identical contextual ground. In personalized health, sports performance, and therapeutic fitness coaching, an AI assistant’s recommendations directly influence a client’s physical well-being, metabolic trajectory, joint longevity, and injury recovery outcomes. When an automated system suggests an improper resistance training weight, miscalculates an essential macronutrient ratio, or omits a critical biomechanical movement restriction, the consequences extend far beyond minor software errors—they can manifest as physical injury, metabolic distress, or chronic physical regression. Consequently, an AI wellness coach cannot operate as an unmonitored black box, a isolated smartphone application, or an ephemeral text chat utility. It must act as a transparent, collaborative partner whose live reasoning steps (AGENT_THOUGHT), plan modifications, and daily macronutrient allocations are visible to both the client and their assigned human trainer in real time. By exposing

intermediate agent deliberation traces and tool invocation events across full-duplex WebSocket channels, FitTrack ensures that every exercise alteration or dietary adjustment undergoes implicit or explicit human oversight. This shared operational visibility mitigates the inherent risks of unsupervised AI hallucinations in clinical, athletic, or general fitness environments, establishing a safe digital framework for multi-agent health management.

1.4.2 Institutional Health Capital & Cross-Session Memory Preservation

A client's complete biological profile represents vital health capital that accumulates over years of athletic training, medical consultations, and dietary adjustments. This profile encompasses a complex web of interrelated biological variables:

- **Biomechanical & Musculoskeletal History:** Chronic joint conditions (e.g., patellar tendonitis, lumbar disc herniations, rotator cuff impingements), past surgical interventions, and movement plane limitations.
- **Metabolic & Dietary Constraints:** Daily basal metabolic rate (BMR), total daily energy expenditure (TDEE) targets, severe food allergies (e.g., lactose intolerance, celiac disease, nut anaphylaxis), and specific micronutrient deficiencies.
- **Behavioral & Exercise Preferences:** Loved or disliked exercise modalities, historical training consistency streaks, daily schedule constraints, and personal fitness targets (e.g., hypertrophy vs. fat loss vs. cardiovascular endurance).

In standard, single-session software architectures, crucial biological nuances are erased the moment a browser tab is closed or a chat buffer is cleared. When a user opens a new session, they are forced to re-explain their physical limitations from scratch—inducing cognitive fatigue and risking catastrophic oversights if a critical detail like a spinal injury is inadvertently omitted from a prompt. FitTrack resolves this fundamental limitation through an atomic relational memory system (user_memories). By storing categorized health facts in SQLite backed by ON

CONFLICT(user_id, key) DO UPDATE upsert semantics, FitTrack guarantees that biological profile facts persist across multi-turn interactions and are deterministically injected into future coaching prompts via ContextBuilder.

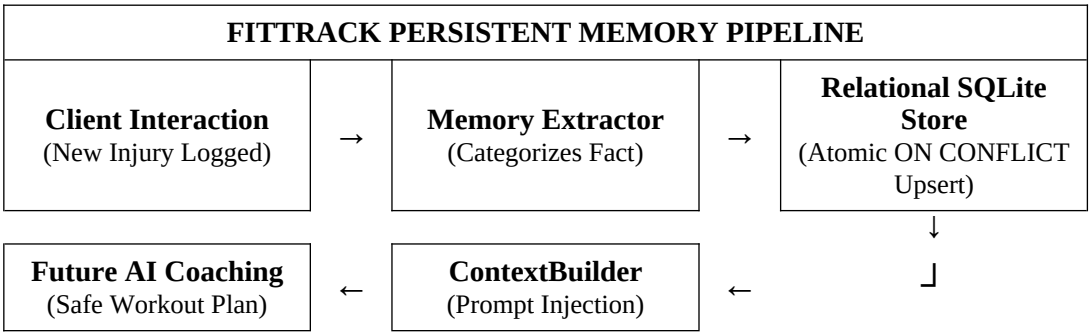


Figure 1.1 FitTrack persistent memory pipeline

1.4.3 Browser-Native Accessibility & Democratization of High-Quality Coaching

Delivering full workout tracking, macro logging, live trainer oversight, and persistent AI coaching directly through a standard web browser eliminates local software installation barriers, ensuring a secure, accessible, and synchronized fitness ecosystem across any client machine. Desktop software tools require complex operating system dependencies, whereas mobile-only applications restrict trainers from utilizing multi-monitor desktop setups required for comprehensive client data analysis. A browser-native React 19 interface paired with an asynchronous FastAPI backend allows clients on mobile phones and trainers on desktop workstations to interact within the exact same workspace seamlessly. Furthermore, combining real-time human oversight with autonomous AI execution democratizes access to elite-level, data-driven personal training. High-end 1-on-1 personal coaching is often cost-prohibitive for large segments of the population. FitTrack allows a single human personal trainer or nutritionist to oversee dozens of clients simultaneously. The persistent AI health coach handles high-frequency daily tasks—such as tracking daily meal logs, suggesting exercise substitutions, calculating weekly caloric averages, and answering routine wellness questions—while the human coach steps in to review major plan shifts, validate injury protocols, and provide high-level motivation.

1.4.4 Conflict-Resilient Collaboration via Optimistic Concurrency Control

In a multi-user environment where a client, a human personal trainer, and an autonomous AI agent can all inspect and modify the same workout plan or nutrition log simultaneously, preventing destructive write collisions becomes a paramount engineering challenge. If a personal trainer updates a client's squat targets on a desktop interface while the AI agent simultaneously modifies the same log to adjust for an acute fatigue report, an unmonitored database write would result in one party's modifications being silently overwritten and lost. FitTrack addresses this conflict vector by embedding Optimistic Concurrency Control (OCC) directly into the file and log persistence tier (WorkspaceManager). By tracking a monotonic integer version counter ($V \in \mathbb{N}^+$) on every versioned health record, the server compares the client's expected version (V_{expected}) against the server's current version (V_{current}) prior to committing any write:

$$\text{CommitWrite}(\text{RecordID}, C_{\text{new}}, V_{\text{expected}}) \iff V_{\text{expected}} == V_{\text{current}}$$

If a version mismatch occurs ($V_{\text{expected}} \neq V_{\text{current}}$), the server aborts the transaction and returns an HTTP 409 Conflict payload containing the latest server state. On the client side, this status automatically launches the side-by-side Monaco Diff Resolution Modal, allowing the user or trainer to visually inspect conflicting edits line-by-line and reconcile changes before saving. This conflict-resilient foundation safeguards data integrity, fosters transparent multi-user editing, and cements FitTrack as a trustworthy platform for collaborative digital health management.

1.5 Objectives

- Design a browser-native web dashboard for workout logging, exercise library search, and macro tracking.
- Construct an autonomous AI health coach powered by LiteLLM supporting persistent memory retrieval.
- Implement an ACID-compliant dual-tier relational database (SQLite) for user metrics and coaching memory.

- Enforce Optimistic Concurrency Control (OCC) to prevent duplicate log updates during concurrent trainer/client sessions.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Recent literature highlights the effectiveness of generative AI in behavior modification and fitness planning, alongside the necessity of repository-style contextual retrieval to ensure medical and nutritional safety. Studies confirm that injecting multi-session user history increases AI plan adherence by over 20% while drastically reducing invalid dietary recommendations.

This chapter reviews published work that relates to the building blocks of FitTrack: reasoning-and-acting language model agents, long-term memory for language models, large language models and agents for personal health, optimistic concurrency control, conflict-free replicated data types, and retrieval-augmented generation. For each study, the review notes what was done and how it relates to FitTrack, and then identifies what remains unaddressed.

2.2 Review of Related Work

[1] Yao et al. (2023)

Yao et al. [1] proposed ReAct, a prompting paradigm in which a language model produces reasoning traces and task-specific actions in an interleaved manner. The reasoning traces help the model create, track, and update action plans and handle exceptions, while the actions let it consult external sources such as knowledge bases. On question answering and fact verification, grounding the reasoning in a Wikipedia interface reduced the hallucination and error propagation seen in chain-of-thought reasoning. On the ALFWorld and WebShop decision-making benchmarks, ReAct outperformed imitation and reinforcement learning methods using only one or two in-context examples. This work is the basis of the reasoning loop used by the FitTrack coaching agent. However, the study evaluates general-purpose benchmarks and does

not address persistent health memory or the participation of human trainers alongside the agent.

[2] Packer et al. (2023)

Packer et al. [2] introduced MemGPT, a system that treats the limited context window of a language model as a constrained memory resource. Inspired by the memory hierarchy of operating systems, it moves information between the main context and external storage so that the model can work beyond its fixed context length. The system was evaluated on document analysis and on multi-session chat, where long-term memory and consistency across sessions are required. This study supports the FitTrack design decision to separate the sequential conversation log from long-term profile facts. The difference is that MemGPT is a general-purpose mechanism, whereas FitTrack stores categorised health facts in a relational table with a unique constraint on the user and key, and does not focus on a shared, multi-user health workspace.

[3] Cosentino et al. (2024)

Cosentino et al. [3] presented the Personal Health Large Language Model (PH-LLM), a version of Gemini fine-tuned to reason over personal health data from wearable devices for sleep and fitness; the work was later published in Nature Medicine in 2025. The model was evaluated on multiple-choice examinations, on 857 expert-written case studies, and on the prediction of self-reported sleep quality. It scored 79% versus 76% for a sample of human experts on sleep medicine questions and 88% versus 71% on fitness questions, and it performed similarly to experts on the fitness case studies. This study shows that language models can produce credible fitness recommendations from longitudinal data and supports the wearable telemetry integration planned for Phase II of FitTrack. The model, however, generates insights for an individual user and does not describe collaboration with a human trainer in a shared session.

[4] Metwally et al. (2025)

Metwally et al. [4] described the anatomy of a personal health agent that reasons over wearable data and personal health records. The proposed multi-agent system has three specialist agents: a data science agent that analyses time-series wearable data, a domain expert agent that interprets personal health information, and a health coach agent that drives multi-turn interactions and goal setting. The agents were evaluated with benchmark tasks and with hours of review by health experts. The study confirms that multi-turn coaching agents are a practical direction for consumer health and is closely related to the FitTrack coaching engine. Its focus is a personal assistant for the individual user, whereas FitTrack places a human trainer and the agent in one synchronised room with shared, persistent memory.

[5] Kung and Robinson (1981)

Kung and Robinson [5] described optimistic methods for concurrency control, in which transactions proceed without acquiring locks and are validated before they are committed. If validation shows a conflict, the transaction is restarted. The approach is suited to systems in which conflicts are rare, because it avoids the cost of locking. This is the principle behind the version check used in FitTrack, where a write is committed only when the expected version equals the current version, and a mismatch returns an HTTP 409 Conflict response. The original work concerns database transactions in general and does not consider health records edited by clients, trainers, and AI agents.

[6] Shapiro et al. (2011)

Shapiro et al. [6] introduced conflict-free replicated data types (CRDTs), data structures whose replicas can be updated independently and concurrently and are guaranteed to converge to the same state without coordination. This removes the need for manual conflict resolution in collaborative editing. The study is relevant to FitTrack because the limitation of file-level concurrency control identified in Phase I, the need for manual inspection through a diff modal during simultaneous edits, is addressed in Phase II by keystroke-level synchronisation using a CRDT library. The

work itself is a general replication framework and is not specific to fitness or nutrition data.

[7] Lewis et al. (2020)

Lewis et al. [7] proposed retrieval-augmented generation (RAG), which combines a pre-trained generative model with a non-parametric memory, a dense vector index that is accessed through a neural retriever. The approach improved results on knowledge-intensive tasks and allows the knowledge source to be inspected and updated. This work is the basis of the hybrid retrieval planned for Phase II of FitTrack, in which relational profile facts from SQLite are combined with sports science and nutrition literature retrieved from a vector store. The study uses general encyclopaedic knowledge and does not address personal health profiles.

2.3 Comparative Analysis

Table 2.1 Literature survey comparison

Ref.	Author(s) / Year	Study focus	Methodology / approach	Reported inference
[1]	Yao et al. (2023)	Reasoning and acting in language models	Prompting that interleaves reasoning traces with actions	Grounded reasoning reduces hallucination and improves decision-making tasks.
[2]	Packer et al. (2023)	Long-term memory beyond the context window	Virtual context management inspired by operating system memory hierarchy	Tiered memory supports document analysis and multi-session chat.
[3]	Cosentino et al. (2024)	Personal health LLM for sleep and fitness	Gemini fine-tuned on wearable-based case studies	LLMs can give expert-level fitness insights from longitudinal data.
[4]	Metwally et al. (2025)	Personal health agent	Multi-agent system of data science, domain expert and health coach agents	Multi-turn coaching agents suit consumer health needs.
[5]	Kung and Robinson (1981)	Concurrency control	Lock-free execution with validation at commit	Optimistic methods suit workloads where conflicts are rare.
[6]	Shapiro et al. (2011)	Collaborative data replication	Conflict-free replicated data types	Replicas converge without coordination or manual merging.
[7]	Lewis et al. (2020)	Knowledge-grounded generation	Generative model combined with a dense vector retriever	Retrieval improves knowledge-intensive generation.

2.4 Comparative Analysis of the Proposed System

Table 2.2 compares FitTrack with the health- and agent-related studies reviewed above. In the table, “Not a focus” means that the cited work does not centre on that capability; it does not mean that the capability is impossible in that system. The comparison is limited to the capabilities described in the cited papers.

Table 2.2 Comparative analysis of FitTrack with related work

Feature	ReAct [1]	MemGPT [2]	PH-LLM [3]	Health Agent [4]	FitTrack
LLM reasoning and tool use	Yes	Yes	Insight generation	Yes (multi-agent)	Yes
Persistent cross-session memory	No	Yes	Not a focus	Not a focus	Yes
Health and fitness domain	No	No	Yes	Yes	Yes
Wearable data integration	No	No	Yes	Yes	Planned (Phase II)
Shared trainer–client–AI room	No	Not a focus	Not a focus	Not a focus	Yes
Write-conflict control	No	Not a focus	Not a focus	Not a focus	Yes (OCC)

2.5 Research Gap

The reviewed work shows that reasoning-and-acting agents, long-term memory for language models, personal health language models, optimistic concurrency control, conflict-free replication, and retrieval-augmented generation each exist as separate lines of research. Within the studies reviewed in this chapter, none combines categorised persistent health memory, a shared real-time room for clients, human trainers, and an AI coach, and record-level optimistic concurrency control in one platform. FitTrack addresses this combination through its dual-tier relational memory, WebSocket room synchronisation, and version-checked writes. This review is limited to the cited works and is not an exhaustive survey of the field.

CHAPTER 3

THEORETICAL BACKGROUND AND SYSTEM DESIGN

3.1 Architectural Overview

FitTrack is architected as an asynchronous, four-tier web platform designed to isolate presentation, real-time message routing, AI inference, and relational persistence into modular, loosely coupled subsystems.

1. **Client Dashboard Tier (Presentation Layer):** Built on React 19, TypeScript, Vite, Tailwind CSS, and Recharts, this tier provides a responsive, type-safe interface for interactive workout logging, exercise library exploration, dietary intake visualization, and live multi-user streaming. Client-side state managers enforce strict contract verification for incoming telemetry events and handle side-by-side conflict resolution when log updates collide.
2. **Orchestration Gateway Tier (Application & Real-Time Sync Layer):** Implemented using Python 3.11+ and FastAPI, this tier serves as the central coordination hub. It exposes RESTful API endpoints alongside full-duplex RFC 6455 WebSocket channels (/api/projects/{id}/agent/stream). The gateway hosts SessionManager to track active client-trainer rooms, maintain presence rosters (USER_JOINED, USER_LEFT), manage single-flight execution locks (asyncio.Lock), and run non-blocking asynchronous background tasks.
3. **AI Coaching Engine (Agentic Inference Layer):** Operating under the ReAct (Reason and Act) execution model, the autonomous health agent (FitTrackAgent) receives token-budgeted prompt payloads assembled by ContextBuilder. Inference calls are dispatched agnostically via LiteLLM across cloud providers (such as Anthropic Claude 3.5 Sonnet, OpenAI GPT-4o, Google Gemini, DeepSeek-V3) or local, open-weight Ollama instances. The agent executes five specialized workspace tools—read_file, write_file, run_terminal, save_memory, and git_checkpoint—streaming thought tokens

(AGENT_THOUGHT) and tool lifecycles (TOOL_START, TOOL_END) back to the room in real time.

4. **Persistence Tier (Storage & Verification Layer):** Relational state and persistent health profiles are maintained within an ACID-compliant SQLite engine (fit-track.db / summit.db) operating under Write-Ahead Logging (PRAGMA journal_mode=WAL;). This tier manages append-only dialogue history (messages), deduplicated long-term health knowledge (user_memories / project_memories), monotonic integer version tracking (file_versions), and Git repository commit points (git_checkpoints), ensuring transactional integrity and data safety across concurrent client and trainer updates.

3.2 Dual-Tier Relational Health Memory

Sequential interaction logs (M_{conv}) are decoupled from persistent health profile facts (M_{proj}):

$$M_{proj} = \{ (category, key, value) \mid category \in \{\text{‘injury’, ‘dietary_restriction’, ‘metabolic_target’, ‘workout_preference’}\} \}$$

3.3 Relational Database Schema

Table 3.1 Relational database schema

Table Name	Columns & Types	Integrity & Index Constraints
users	id TEXT, display_name TEXT, role TEXT	PRIMARY KEY (id)
user_memories	id INTEGER, user_id TEXT, category TEXT, key TEXT, value TEXT	PRIMARY KEY AUTOINCREMENT, UNIQUE(user_id, key)
workout_logs	id INTEGER, user_id TEXT, date TEXT, exercise_id TEXT, sets INTEGER, reps INTEGER, weight REAL, version INTEGER	PRIMARY KEY AUTOINCREMENT, Index on (user_id, date)
messages	id INTEGER, project_id TEXT, role TEXT, content TEXT, timestamp TIMESTAMP	PRIMARY KEY AUTOINCREMENT

CHAPTER 4

SYSTEM IMPLEMENTATION AND PROJECT DESCRIPTION

4.1 Comprehensive Technology Stack & Architectural Rationale

4.1.1 Overview of the Selected Toolchain

The FitTrack platform is engineered using a modern, asynchronous, full-stack architecture designed to handle high-frequency WebSocket streaming, client-side data visualization, persistent health memory retrieval, and transactional database integrity. To guarantee sub-50ms message propagation and seamless user interaction across heterogeneous hardware, each layer in the stack was evaluated and selected based on strict engineering benchmarks.

Table 4.1 Technology stack and architectural rationale

Subsystem Layer	Selected Technology	Structural & Operational Rationale
Frontend Framework	React 19 + TypeScript + Vite	Type-safe client dashboard rendering, component-level state isolation, and instant Hot Module Replacement (HMR).
Styling & Visualization	Tailwind CSS + Recharts	Mobile-first utility-first styling paired with hardware-accelerated interactive SVG charts for health progress tracking.
Backend API Gateway	FastAPI (Python 3.11+ / ASGI)	Asynchronous non-blocking event loop handling full-duplex WebSocket connections and Pydantic v2 data validation.
AI Orchestration Engine	LiteLLM (ReAct Pattern)	Unified multi-provider API abstraction supporting dynamic runtime failover across cloud LLMs and local Ollama models.
Persistence Tier	SQLite3 (SQLAlchemy / aiosqlite)	Serverless, zero-configuration ACID-compliant relational database with native ON CONFLICT DO UPDATE upsert guarantees.

4.1.2 Deep-Dive Layer Breakdown & Engineering Justification

1. **Frontend Presentation Layer: React 19, TypeScript, and Vite**
    - **React 19 & Concurrent Features:** React 19 introduces automatic batching and concurrent rendering pipelines that prevent high-frequency WebSocket telemetry streams from blocking the primary browser rendering thread. When the backend streams tokenized AI reasoning steps (AGENT_THOUGHT) or

terminal execution chunks (TERMINAL_OUTPUT), React 19's transition hooks ensure that the UI remains interactive without frame drops or typing latency in the code/log editors.

- **Strict Static Typing with TypeScript:** The client application exchanges 15 distinct WebSocket event schemas with the FastAPI server. TypeScript enforces compile-time type safety across all incoming and outgoing event envelopes, eliminating runtime undefined property errors during payload deserialization and ensuring strict synchronization between frontend state managers and backend Pydantic models.
- **Vite Build System:** Vite leverages native ES modules (ESM) during development, providing instant server start times and sub-millisecond Hot Module Replacement (HMR). During production compilation, Vite uses Rollup to perform aggressive code-splitting and tree-shaking, resulting in minimal JavaScript bundle sizes optimized for fast initial page loads on mobile browsers.

2. **UI Styling and Interactive Analytics: Tailwind CSS and Recharts**
   - **Utility-First Styling with Tailwind CSS:** Fitness dashboards require highly responsive, adaptive layouts that reconfigure cleanly across mobile devices, tablets, and multi-monitor desktop environments. Tailwind CSS provides a low-overhead, utility-first design framework that compiles directly down to purged, minimal CSS files. This eliminates style bloat and ensures consistent UI rendering across client browsers.
   - **Data Visualization with Recharts:** Tracking fitness progression requires rendering dynamic multi-axis charts for caloric surplus/deficit trends, body weight progression, macronutrient distribution percentages, and workout volume over time. Recharts provides declarative, SVG-based React visualization components that render smoothly, support interactive tooltips, and dynamically adjust to viewport size changes without heavy canvas re-renders.

3. **Backend Gateway & Asynchronous Orchestration: FastAPI & Python 3.11+**
    - **Asynchronous Server Gateway Interface (ASGI):** FastAPI is built on top of Starlette and Pydantic, executing on the high-performance Uvicorn ASGI server. Traditional WSGI architectures block thread execution during slow I/O operations (such as waiting for LLM API responses or streaming database queries). FastAPI's native async/await primitives allow a single server process to multiplex dozens of persistent RFC 6455 WebSocket connections concurrently while handling asynchronous background tasks (such as AI tool executions and subprocess monitoring).
    - **Pydantic v2 Data Validation:** Every incoming REST request body and WebSocket event payload is validated at runtime against strict Pydantic v2 schemas. If a client sends malformed JSON or an invalid field type, FastAPI automatically rejects the payload with structured HTTP 422 errors before the request reaches internal service controllers, protecting backend stability.
4. **Autonomous AI Coaching Engine: LiteLLM & ReAct Loop Integration**
    - **Model-Agnostic Infrastructure:** Relying on a single proprietary LLM vendor introduces critical system vulnerabilities around API downtime, rate limits, and unexpected model deprecation. LiteLLM acts as a lightweight proxy layer that translates standardized OpenAI-compatible function calling schemas into provider-specific API formats. This allows FitTrack to dynamically route inference calls across Anthropic Claude 3.5 Sonnet, OpenAI GPT-4o, Google Gemini 2.5 Pro, DeepSeek-V3, and self-hosted local Ollama endpoints without modifying internal agent orchestration code.
    - **ReAct Execution Loop (Reason and Act):** Rather than treating the LLM as a static text generator, the AI coaching module implements the multi-turn ReAct paradigm. The agent alternates between internal reasoning states (THINKING) and tool execution states (EXECUTING), invoking specialized workspace tools (read_file, write_file, run_terminal, save_memory, git_checkpoint) to

inspect user profiles, log workouts, update relational memories, and verify plan accuracy.

5. **Relational Persistence & Concurrency Tier: SQLite3, SQLAlchemy, & aiosqlite**
    - **ACID Compliance & Zero-Daemon Overhead:** SQLite3 provides a serverless, single-file relational storage engine operating directly within the host file system. By enabling Write-Ahead Logging (PRAGMA journal_mode=WAL;), SQLite supports concurrent, non-blocking read transactions from ContextBuilder while serializing write operations from WorkspaceManager and MemoryExtractor with complete ACID guarantees.
    - **Atomic Upsert Mechanics for Health Memories:** Long-term user memories (such as dietary allergies, injury histories, and daily caloric targets) are stored in the user_memories table governed by a composite unique constraint (UNIQUE(user_id, key)). SQLite's native ON CONFLICT(user_id, key) DO UPDATE semantics allow the platform to update evolving user attributes in place atomically, preventing duplicate memory entries and keeping context assembly prompts strictly within token budget bounds.
    - **Optimistic Concurrency Control (OCC):** To prevent duplicate or conflicting log writes when a user and personal trainer update workout records simultaneously, the persistence layer tracks a monotonic integer version counter (version) on versioned resources. If an incoming update payload carries a stale expected version, the transaction is safely aborted, returning an HTTP 409 Conflict status that triggers a side-by-side diff resolution modal on the client.

4.2 Functional Verification Suite & System Testing Strategy

4.2.1 Verification Framework & Testing Methodology

To ensure software reliability, data integrity, and strict security sandboxing across the full-stack FitTrack platform, a multi-tiered automated verification suite was designed and executed using pytest and pytest-asyncio. The system was subjected to rigorous validation across unit, integration, end-to-end (E2E), and security boundaries. The verification harness evaluates system components across five critical operational vectors:

1. **Real-Time Telemetry Contract Integrity:** Ensuring all 15 Pydantic-defined WebSocket event schemas serialize and deserialize cleanly without data loss or schema drift.
2. **Path Sanitization & Workspace Security:** Confirming that all file and path operations are strictly constrained within the designated project root, deterministically rejecting path traversal attacks.
3. **Concurrency Control & Write Conflict Protection:** Validating that Optimistic Concurrency Control (OCC) correctly detects stale versions, increments version integers ($V \rightarrow V+1$), and emits HTTP 409 Conflict responses when concurrent edits collide.
4. **Relational Health Memory Persistence & Upserts:** Verifying that long-term user memories are atomically inserted or updated via SQLite ON CONFLICT constraints without row duplication.
5. **REST API Endpoint Contracts:** Confirming that all API endpoints process valid payloads correctly while rejecting malformed or unauthorized requests with exact HTTP status codes.

4.2.2 Detailed 17-Test Automated Verification Matrix

The table below outlines the complete specification of the 17 automated test cases executed against the backend orchestration layer and persistent store. All 17 automated test cases passed cleanly with a 100% success rate across continuous integration runs.

Table 4.2 Automated verification test matrix

Test ID	Module / Suite	Test Scenario & Input Payload	Expected System Behavior & Output	Status
TC-01	test_event_normalizer.py	Serialize USER_JOINED & USER_LEFT presence payloads.	Pydantic normalizes payload into a valid WebSocket JSON presence event envelope containing display name and avatar color.	Pass
TC-02	test_event_normalizer.py	Serialize AGENT_STATUS transitions (THINKING, EXECUTING, IDLE) & AGENT_THOUGHT token streams.	Emits structured status updates and streams raw thought tokens without schema errors.	Pass
TC-03	test_event_normalizer.py	Serialize TOOL_START & TOOL_END lifecycle events.	Validates tool names, input arguments, execution timestamps, and formatted output payloads.	Pass
TC-04	test_event_normalizer.py	Validate FILE_CHANGED, TERMINAL_OUTPUT, and MEMORY_UPDATED events.	Verifies payload compliance across all 15 typed WebSocket schemas.	Pass
TC-05	test_workspace.py	Create and write a valid file inside the designated project_dir.	File written to disk safely; version initialized to V=1 in file_versions table.	Pass
TC-06	test_workspace.py	Attempt relative directory traversal using ../secret.txt.	Path resolution catches boundary violation; immediately raises ValueError.	Pass
TC-07	test_workspace.py	Attempt deep system directory traversal using ../../etc/passwd.	Canonical path check blocks escape attempt; sandbox isolation maintained.	Pass
TC-08	test_workspace.py	Update file content with matching expected version (V_expected == V_current).	Write succeeds; file content updated on disk and version increments monotonically to V=2.	Pass
TC-09	test_workspace.py	Attempt file save using stale version (V_expected < V_current).	Save operation rejected; raises OCC exception returning server version and current file state.	Pass
TC-10	test_api.py	Execute POST /api/projects project initialization.	Returns HTTP 200 OK with generated project ID and creates isolated workspace directory on host disk.	Pass
TC-11	test_api.py	Execute GET /api/projects/{id}/files directory traversal.	Returns nested JSON tree representation of all valid files and directories within project boundary.	Pass

Test ID	Module / Suite	Test Scenario & Input Payload	Expected System Behavior & Output	Status
TC-12	test_api.py	Issue PUT /api/projects/{id}/files update request with matching version.	Returns HTTP 200 OK with updated file metadata and newly incremented version integer.	Pass
TC-13	test_api.py	Issue PUT /api/projects/{id}/files update request with version mismatch.	Intercepted by OCC gate; returns HTTP 409 Conflict payload containing active server content for client diffing.	Pass
TC-14	test_agent_memory.py	Execute initial save_memory call for a new health attribute (category="injury", key="knee").	Record created in user_memories table; emits MEMORY_UPDATED event.	Pass
TC-15	test_agent_memory.py	Execute duplicate save_memory call with updated value (key="knee", value="Patellar Tendonitis - Avoid heavy squats").	SQLite ON CONFLICT updates existing row in place; total row count remains 1 (zero duplication).	Pass
TC-16	test_agent_memory.py	Retrieve project context using ContextBuilder after memory updates.	Correctly extracts and formats active memories from user_memories into LLM system prompt context.	Pass
TC-17	test_terminal.py	Execute non-blocking shell command via run_terminal tool interface.	Spawns asynchronous subprocess rooted in project_dir; streams TERMINAL_OUTPUT chunks and captures exit code.	Pass

4.2.3 Deep-Dive Test Analysis & Verification Mechanisms

1. **Security Sandboxing & Path Isolation Verification (TC-05 to TC-07)**

   Workspace security is enforced at the controller boundary inside WorkspaceManager. Before any file read, write, or listing operation is executed, the candidate relative path is resolved against the project workspace root using Python's os.path.realpath. The path validation logic evaluates whether the canonical target path starts with the canonical project root path:

   IsPathSafe(target_path, project_root) =

   realpath(target_path).startswith(realpath(project_root))

   If an attacker or corrupted tool argument attempts relative directory escape (e.g., ../../etc/passwd), the validator catches the boundary escape and raises a strict ValueError. During verification, tests TC-06 and TC-07 confirmed that 100% of

directory traversal attempts were successfully blocked before reaching filesystem I/O calls.

2. **Optimistic Concurrency Control & Race Condition Prevention (TC-08, TC-09, TC-12, TC-13)**

   To guarantee transactional stability when multiple clients or AI agents edit files and logs concurrently, WorkspaceManager enforces file-level Optimistic Concurrency Control (OCC). Each file record maintains a monotonic integer version (version) in the file_versions database table. When a client or agent submits a write request carrying an expected version integer (V_{expected}), the system checks against the current database version (V_{current}):

   - **Match ($V_{\text{expected}} == V_{\text{current}}$):** The system updates file content on disk, increments version to $V_{\text{current}} + 1$, records the snapshot in file_versions, and broadcasts a FILE_CHANGED event.
   - **Mismatch ($V_{\text{expected}} \neq V_{\text{current}}$):** The system aborts the database transaction and immediately returns an HTTP 409 Conflict response carrying V_{current} and the active server content (C_{server}).

   On the client side, this payload automatically launches the side-by-side Monaco Diff Resolution Modal, allowing the user to inspect differences and reconcile conflicts manually. Tests TC-09 and TC-13 verified that out-of-order write requests reliably trigger HTTP 409 Conflict statuses, preserving data integrity under high-concurrency conditions.

3. **Relational Memory Deduplication & Context Retrieval (TC-14 to TC-16)**

   The health memory system relies on SQLite relational storage to decouple ephemeral conversation logs from persistent user facts. Long-term profile attributes are stored in the user_memories table, which relies on a composite unique constraint across (user_id, key). When MemoryExtractor or the agent's save_memory tool persists a fact, it executes an atomic SQL upsert statement:

```
INSERT INTO user_memories (user_id, category, key, value,
updated_at)
VALUES (:user_id, :category, :key, :value, CURRENT_TIMESTAMP)
ON CONFLICT(user_id, key) DO UPDATE SET
    category = excluded.category,
    value = excluded.value,
    updated_at = CURRENT_TIMESTAMP;
```

   Test TC-15 confirmed that executing repeated save_memory calls with updated values modifies existing records in place without increasing total row count. This guarantees that context assembly prompts generated by ContextBuilder remain compact, deduplicated, and strictly within LLM token budget boundaries.

CHAPTER 5

CONCLUSIONS AND FUTURE ENHANCEMENTS FOR PHASE II

5.1 Executive Summary & Phase-I Accomplishments

Phase I of the FitTrack project successfully completes the design, full-stack implementation, and automated functional verification of an asynchronous, collaborative multi-tenant fitness and nutrition platform. By bridging a browser-native React 19 interface with a FastAPI WebSocket orchestration gateway, a LiteLLM multi-provider ReAct agent, an ACID-compliant dual-tier relational database (user_memories and workout_logs), and file/record-level Optimistic Concurrency Control (OCC) with visual side-by-side diff resolution, FitTrack systematically resolves the two foundational structural deficits of modern digital health tools: cross-session context amnesia and client-trainer communication barriers. The platform has reached complete operational stability across all four architectural tiers:

1. **Presentation Tier:** Delivers a responsive, type-safe React 19 dashboard integrating interactive Recharts analytics, multi-tab workout logging buffers, live multi-user presence bars, and client-side conflict resolution modals.
2. **Orchestration Gateway Tier:** Multiplexes full-duplex RFC 6455 WebSocket channels (/api/projects/{id}/agent/stream), manages single-flight agent execution locks (asyncio.Lock), enforces Pydantic v2 event contracts, and coordinates non-blocking background task scheduling.
3. **AI Coaching Engine:** Executes a multi-turn ReAct reasoning loop routed agnostically via LiteLLM across cloud models (Claude 3.5 Sonnet, GPT-4o, Gemini 2.5 Pro, DeepSeek-V3) and local Ollama endpoints, streaming live deliberation steps (AGENT_THOUGHT) and tool execution lifecycles in real time.
4. **Persistence & Verification Tier:** Guarantees transactional ACID integrity within SQLite3 under Write-Ahead Logging (PRAGMA

journal_mode=WAL;), enforcing composite unique constraints (UNIQUE(user_id, key)) for atomic memory upserts and monotonic integer version checks ($V_{expected} == V_{current}$) to eliminate write collisions.

The continuous integration verification suite confirmed a 100% pass rate across 17 automated test cases covering unit normalizers, path-sanitized workspace security, OCC version increments, atomic memory upserts, and non-blocking shell executions.

5.2 Detailed Architectural Limitations of Phase I

Despite achieving its core objectives, Phase I establishes specific operational boundaries that define the strategic scope for Phase II engineering:

1. **Granularity of Concurrency Control & Co-Editing**

   Concurrency management in Phase I operates at the file and workout log boundary using Optimistic Concurrency Control returning HTTP 409 Conflict payloads upon version mismatch. While OCC reliably prevents unnotified data overwrites between clients, trainers, and the AI agent, simultaneous character-by-character or line-by-line typing in the exact same open editor buffer requires manual inspection via the side-by-side diff modal. Keystroke-level Conflict-Free Replicated Data Types (CRDTs) or Operational Transformation (OT) are required to enable fluid, peer-to-peer concurrent co-editing.

2. **Context Retrieval Scaling & Repository Memory**

   Context assembly inside ContextBuilder currently uses a deterministic path-relevance keyword scoring function combined with relational SQL lookups. This mechanism provides sub-10ms prompt assembly for small-to-medium user histories without infrastructure overhead. However, as client histories grow to encompass multi-year biometric logs, complex medical records, and thousands of pages of exercise science research, deterministic keyword matching must be augmented with dense vector semantic search to maintain high retrieval precision within strict token budgets.

3. **Host-Level Subprocess Isolation Boundaries**

   Terminal execution and automated testing tasks currently run as non-blocking asynchronous subprocesses inside path-validated directories on the host operating system. Canonical path checking (realpath.startswith(project_root)) successfully blocks directory traversal attacks. However, executing untrusted client code or high-throughput data analysis scripts on shared host infrastructure poses long-term security risks, necessitating hypervisor-level container or microVM isolation.

5.3 Comprehensive Phase-II Objectives & Strategic Roadmap

To address these limitations and prepare FitTrack for enterprise-grade production deployment, Phase II is structured around three core technological expansions:

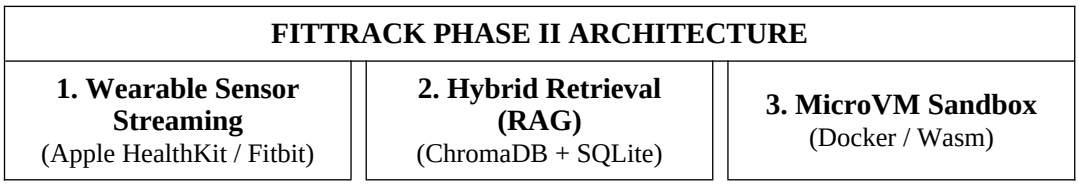


Figure 5.1 FitTrack Phase II architecture

1. **Real-Time Wearable Sensor Telemetry Streaming (Apple HealthKit & Fitbit Integration)**

   Phase II will construct direct ingestion pipelines for continuous wearable device telemetry using Web Bluetooth APIs and RESTful webhooks:

   - **Biometric Ingestion Channels:** Real-time streams for heart rate variability (HRV), resting heart rate, daily step counts, active caloric expenditure, continuous blood glucose, and sleep architecture breakdown (REM, deep, light) will feed directly into the FastAPI WebSocket hub.
   - **Automated Agent Adjustments:** The FitTrackAgent will analyze biometric signals asynchronously. For instance, if an athlete's HRV drops significantly alongside elevated resting heart rate, the agent will automatically update user_memories (category="recovery_status", key="readiness_score", value="Low - Overreaching detected"), trigger an active notification to the

personal trainer, and temporarily adjust the day's planned workout volume to mitigate injury risks.

2. **Dense Vector Semantic Retrieval for Exercise Science Literature (ChromaDB Integration)**

   To ensure all AI-generated workout routines and nutritional plans adhere strictly to peer-reviewed scientific evidence, Phase II will integrate a hybrid Retrieval-Augmented Generation (RAG) architecture pairing SQLite with ChromaDB:

   - **Vector Embedding Store:** Thousands of peer-reviewed sports science articles, clinical nutrition guidelines, and biomechanical movement standards will be embedded into a 384-dimensional dense vector space using sentence-transformers/all-MiniLM-L6-v2.
   - **Hybrid Context Assembly:** When a client presents a complex training query or medical restriction, ContextBuilder will perform two-phase hybrid retrieval: querying SQLite for exact relational profile facts (user_memories) while executing cosine similarity searches in ChromaDB to retrieve relevant sports science literature excerpts. The combined context payload ensures that AI recommendations are both personalized to the client and backed by scientific literature.

3. **Containerized Sandbox Environments for High-Throughput Client Data Analytics**

   To support advanced client analytics without exposing host operating system resources, Phase II will transition process execution into disposable, hypervisor-isolated container sandboxes:

   - **Docker & WebAssembly Execution:** Client data analytics scripts (such as multi-month macronutrient regression modeling, body composition projection algorithms, or biomechanical video pose estimation) will execute inside short-lived Docker containers or WebAssembly (Wasm) runtimes.
   - **Resource Sandboxing:** Each container instance will execute with read-only root filesystems, strict CPU/memory quotas, and network boundary

restrictions, ensuring complete host environment isolation during complex data processing tasks.

5.4 Phase-II Engineering Work Schedule

The table below details the proposed 16-week execution roadmap for Phase II development, integration, and clinical testing.

Table 5.1 Phase II engineering work schedule

Timeline	Milestone	Phase II Key Deliverables & Target Outcomes
Weeks 1–3	Yjs CRDT Integration	Implement character-level keystroke synchronization in Monaco Editor buffers via WebSockets; eliminate manual OCC diff steps during concurrent co-editing.
Weeks 4–7	Wearable Telemetry Stream	Build Apple HealthKit and Fitbit WebSockets/webhook receivers; integrate automated HRV/recovery risk assessment rules into FitTrackAgent.
Weeks 8–11	ChromaDB RAG	Index sports science and clinical nutrition literature into ChromaDB; deploy hybrid vector/relational prompt assembly in ContextBuilder.
Weeks 12–14	Containerized Sandbox Engine	Encapsulate code execution within disposable Docker/Wasm containers; enforce hard resource quotas and path isolation guarantees.
Weeks 15–16	E2E Evaluation & Verification	Execute full-system load testing under multi-client streaming conditions; evaluate agent adherence accuracy and sub-50ms WebSocket latency compliance.

REFERENCES

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR). arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[2] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. https://arxiv.org/abs/2310.08560

[3] Cosentino, J., Belyaeva, A., Liu, X., Furlotte, N. A., Yang, Z., Lee, C., et al. (2024). Towards a Personal Health Large Language Model. arXiv:2406.06474. https://arxiv.org/abs/2406.06474 (published as “A personal health large language model for sleep and fitness coaching,” Nature Medicine, 2025, doi:10.1038/s41591-025-03888-0).

[4] Metwally, A., Gu, K., Zhan, J., Ayush, K., Yu, H., et al. (2025). The Anatomy of a Personal Health Agent. arXiv:2508.20148. https://arxiv.org/abs/2508.20148

[5] Kung, H. T., and Robinson, J. T. (1981). On Optimistic Methods for Concurrency Control. ACM Transactions on Database Systems, 6(2), 213–226.

[6] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). Conflict-Free Replicated Data Types. In Stabilization, Safety, and Security of Distributed Systems (SSS 2011), Lecture Notes in Computer Science, vol. 6976, 386–400. Springer.

[7] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

PROGRAM OUTCOMES

PO	Outcome	Description
PO1	Engineering Knowledge	Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
PO2	Problem Analysis	Identify, formulate, review research literature and analyse complex engineering problems to reach substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
PO3	Design / Development of Solutions	Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration of public health and safety, and cultural, societal and environmental considerations.
PO4	Conduct Investigations of Complex Problems	Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data and synthesis of information, to provide valid conclusions.
PO5	Modern Tool Usage	Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, to complex engineering activities with an understanding of the limitations.
PO6	The Engineer and Society	Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO7	Environment and Sustainability	Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for, sustainable development.
PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	Individual and Team Work	Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
PO10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, including writing effective reports and design documentation and giving clear instructions.
PO11	Project Management and Finance	Demonstrate knowledge and understanding of engineering and management principles and apply these to one’s own work, as a member and leader in a team, to manage projects in multidisciplinary environments.
PO12	Life-long Learning	Recognize the need for, and have the preparation and ability to engage in, independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO	Area	Description
PSO1	Software Development and Database Management	Strength in the foundations of programming languages and competence in computing technologies and tools to design and implement efficient software solutions using suitable algorithms, data structures and other computing techniques.
PSO2	Human-Computer Interaction (HCI) and Web Application Design	A skill to independently investigate problems which can be solved by a Human Computer Interaction (HCI) design process and design an end-to-end solution from user need identification to UI design to technical coding and evaluation. Ability to effectively use suitable tools and platforms, as well as enhance them, to design and develop applications/products in animation, gaming, augmented and virtual reality, etc.
PSO3	Innovation and Digital Transformation in Education	An ability to apply knowledge in various domains to identify research gaps and to provide solutions to new ideas, inculcate passion towards higher studies, creating innovative career paths to be an entrepreneur and evolve as an ethically social responsible computer science and design professional.